Introduction to Artificial Intelligence

Semester I, 2025-26

Reinforcement Learning

Rohan Paul

Outline

- Last Class
  - Markov Decision Processes
- This Class
  - Reinforcement Learning
- Reference Material
  - Please follow the notes as the primary reference on this topic. Supplementary reading on topics covered in class from AIMA Ch 21 sections 21.1 – 21.4.

Acknowledgement

These slides are intended for teaching purposes only. Some material has been used/adapted from web sources and from slides by Doina Precup, Dorsa Sadigh, Percy Liang, Mausam, Parag, Emma Brunskill, Alexander Amini, Dan Klein, Anca Dragan, Nicholas Roy and others.

The problem of learning policies (good behaviour) through interaction with the environment

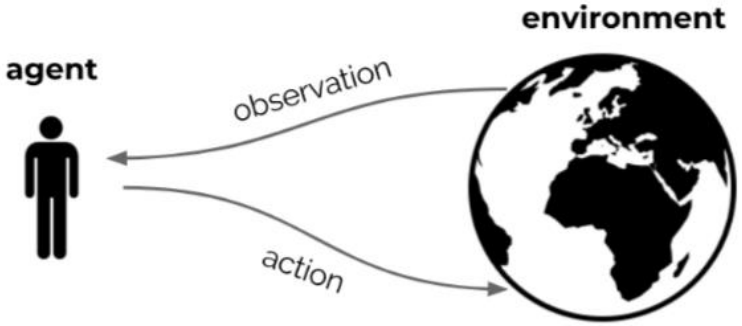
Reinforcement Learning (RL)

- People and animals learn by interacting with the environment
  - It is active rather than passive.
  - Interactions are often sequential — future interactions can depend on earlier ones

Biological motivation

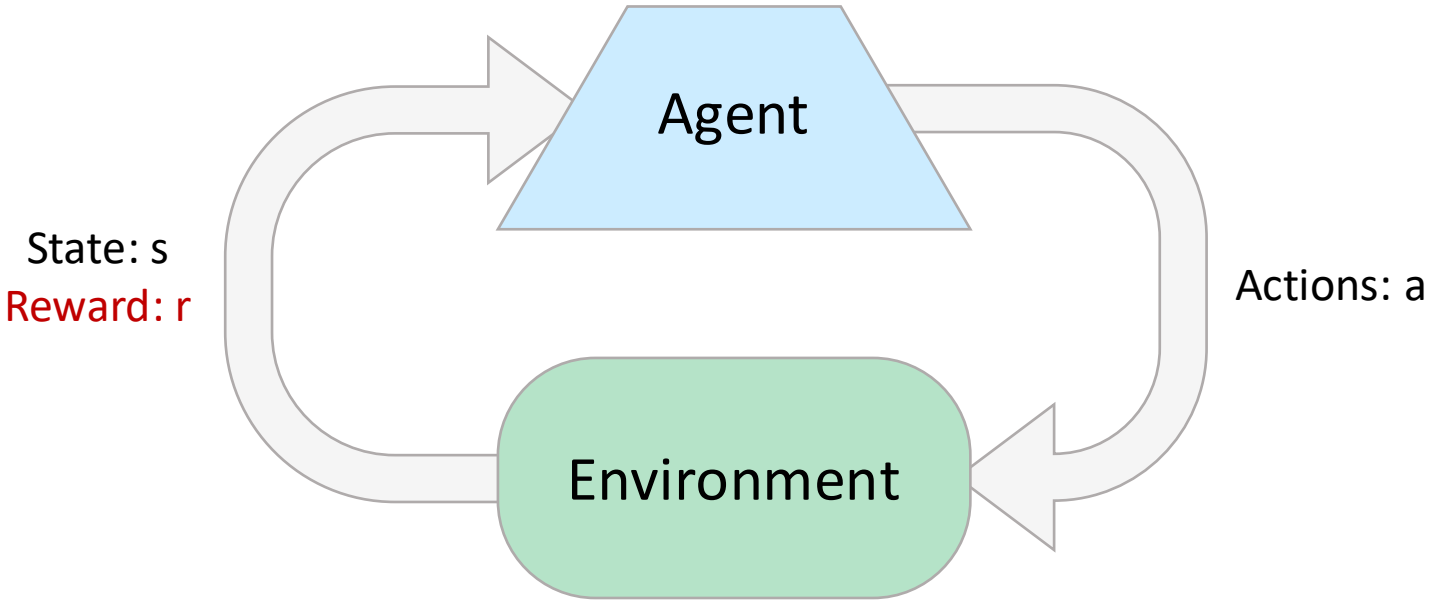
- Reinforcement Learning

Wikipedia: reinforcement learning is an area of machine learning inspired by behavioural psychology, concerned with how software agents ought to take actions in an environment so as to maximize some notion of cumulative reward



Goal: optimise sum of rewards, through repeated interaction

Reinforcement Learning: Setup



Key characteristic of reinforcement learning

- **Only evaluative feedback present.**
- **The agent takes and action and is provided feedback (reward).**
- **The agent is not told which action it should take in a state.**

- Receive feedback in the form of rewards. Agent's utility is defined by the reward function
- Must (learn to) act so as to maximize expected rewards
- All learning is based on observed samples of outcomes!

Reinforcement Learning: Formal Setup

- Formal Definition
  - States: $s \in S$
  - Actions: $a \in A$
  - Rewards: $r \in \mathbb{R}$
  - ~~Transition model: $\Pr(s_t|s_{t-1}, a_{t-1})$~~
  - ~~Reward model: $\Pr(r_t|s_t, a_t)$~~
  - Discount factor: $0 \leq \gamma \leq 1$
  - Horizon (i.e., # of time steps): h

} unknown

I take an action but I do not know in functional form which state I will be in and whether it will take me closer to my goal.

I do not have a-priori access to the transition or the reward model.

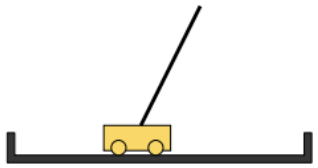
Think of this as an incomplete MDP.

- Goal: find optimal policy $\pi^* = argmax_\pi \sum_{t=0}^{h} \gamma^t E_\pi[r_t]$

The AI algorithm will interact with the environment and learn what are good actions and what are bad actions based on feedback! It's the kind of learning that we often do.

Examples of RL Use Cases

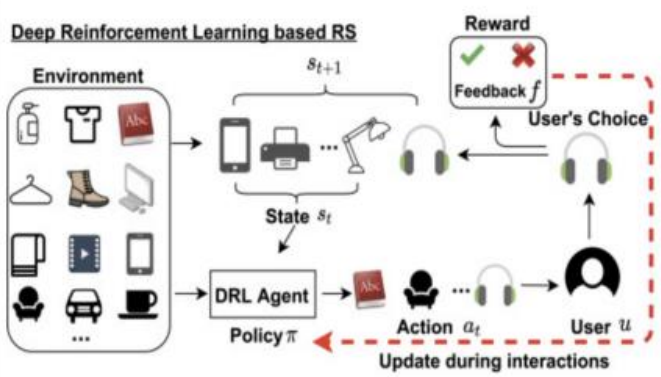
- State: $x(t), x'(t), \theta(t), \theta'(t)$
- Action: Force F
- Reward: 1 for any step where pole balanced



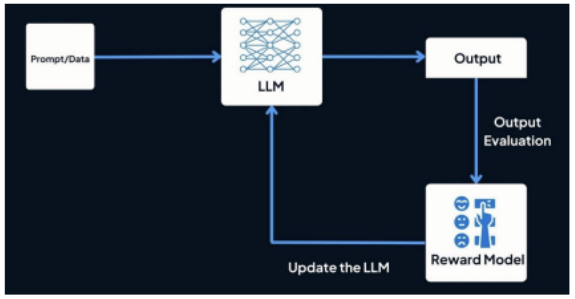
Problem: Find $\pi: S \rightarrow A$ that maximizes rewards

[Example of a complex task: https://www.youtube.com/watch?v=kCZc9SJYLno](https://www.youtube.com/watch?v=kCZc9SJYLno)

- **Agent:** recommender system
- **Environment:** user
- **State:** context, past recommendations and feedback
- **Action:** recommended item
- **Reward:** value of user feedback



- **Agent:** system
- **Environment:** user
- **State:** history of past utterances
- **Action:** system utterance
- **Reward:** task completion, human feedback



Credit: https://www.twine.net/blog/what-is-reinforcement-learning-from-human-feedback-rlhf-and-how-does-it-work/

- **Agent:** trading software
- **Environment:** other traders
- **State:** price history
- **Action:** buy/sell/hold
- **Reward:** amount of profit

Example: how to purchase a large # of shares in a short period of time without affecting the price

Examples: Learning to Walk

Initial

[Kohl and Stone, ICRA 2004]

Examples: Learning to Walk

Training

[Kohl and Stone, ICRA 2004]

Examples: Learning to Walk

Finished

[Kohl and Stone, ICRA 2004]

Different RL Approaches

RL agents may or may not estimate the following components:

- Model: $\Pr(s'|s,a)$, $\Pr(r|s,a)$
  - Environment dynamics and rewards
- Policy: $\pi(s)$
  - Agent action choices
- Value function: $V(s)$
  - Expected total rewards of the agent policy

Categorization of RL Approaches

Value based
- No policy (implicit)
- Value function

Policy based
- Policy
- No value function

Actor critic
- Policy
- Value function

Model based
- Transition and reward model

Model free
- No transition and no reward model (implicit)

Online RL
- Learn by interacting with environment

Offline RL
- No environment
- Learn only from saved data

Classical Techniques for RL: Model based estimation

Model-Based Reinforcement Learning

- Model-Based Idea
  - Learn an approximate model (R(), T()) based on experiences
  - Compute the value function using the learned model (as if it were correct).

- Step 1: Learn an empirical MDP model
  - Count outcomes s' for each s, a
  - Normalize to give an estimate of $\hat{T}(s, a, s')$
  - Discover each $\hat{R}(s, a, s')$ when we experience (s, a, s')

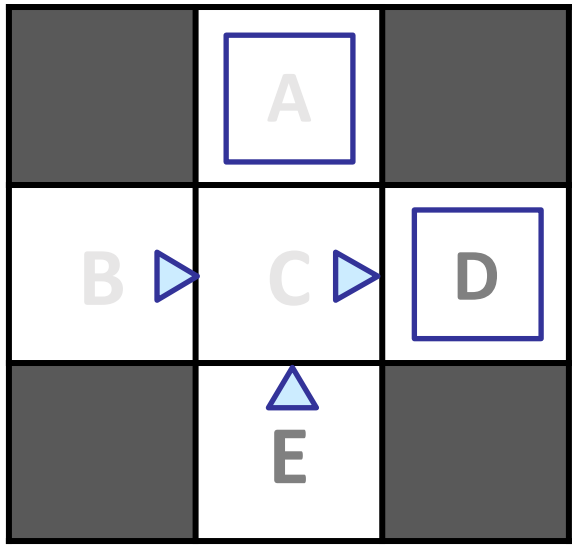
- Step 2: Solve the learned MDP
  - For example, use value iteration, to obtain the final policy.
  - Plug in the estimated T and R in the following equation:

$$\pi^*(s) = \arg\max_a \sum_{s'} \hat{T}(s, a, s')\hat{R}(s, a, s') + \gamma V^*(s')]$$

*Note: going through an intermediate stage of learning the model. In contrast, "**model-free**" approaches do not learn the intermediate model.*

Example: Model-Based Learning

Input Policy π



Assume: $\gamma = 1$

Observed Episodes (Training)

Episode 1

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 2

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 3

E, north, C, -1
C, east, D, -1
D, exit, x, +10

Episode 4

E, north, C, -1
C, east, A, -1
A, exit, x, -10

Learned Model

$\hat{T}(s, a, s')$

T(B, east, C) = 1.00
T(C, east, D) = 0.75
T(C, east, A) = 0.25
…

$\hat{R}(s, a, s')$

R(B, east, C) = -1
R(C, east, D) = -1
R(D, exit, x) = +10
…

Recap: : Model-Based vs Model-Free Estimation

Goal: Compute expected age of students

Known P(A)

$$E[A] = \sum_a P(a) \cdot a \quad = 0.35 \times 20 + \ldots$$

Without P(A), instead collect samples $[a_1, a_2, \ldots a_N]$

Unknown P(A): "Model Based"

$$\hat{P}(a) = \frac{\text{num}(a)}{N}$$

$$E[A] \approx \sum_a \hat{P}(a) \cdot a$$

Eventually, we learn the right model which gives the right estimates.

Unknown P(A): "Model Free"

$$E[A] \approx \frac{1}{N} \sum_i a_i$$

Bypass the model construction. Averaging works because the samples appear with the right frequencies.

Model-Based vs. Model Free RL

Model-based RL:

- Explore environment and learn model, T=P(**s**'|**s**,**a**) and R(**s**,**a**), (almost) everywhere. Use model to plan a policy (solving the MDP)
- Suitable when the state-space is manageable

Model-free RL:

- Do not learn a model; learn value function or policy directly
- Suitable when the state space is large

Next, we look at a model free approach to RL

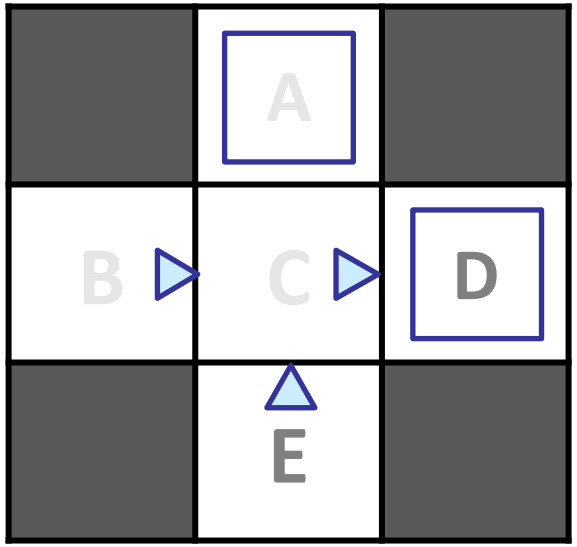
Classical Techniques for RL: Monte Carlo estimation

Monte Carlo Methods

- Learning the state-value function for a given policy
  - What is the value of a state?
    - Expected return – expected cumulative future discounted reward
- Key Idea
  - Sample trajectories from the world directly and estimate a value function without a model
  - Simply average the returns observed after visits to that state.
  - As more returns are observed the average should converge to the expected value.
  - Each occurrence of a state in an episode is a called a visit to the state.

Toy Example: Monte Carlo Method

Input Policy π



Assume: $\gamma = 1$

Observed Episodes (Training)

Episode 1

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 2

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 3

E, north, C, -1
C, east, D, -1
D, exit, x, +10

Episode 4

E, north, C, -1
C, east, A, -1
A, exit, x, -10

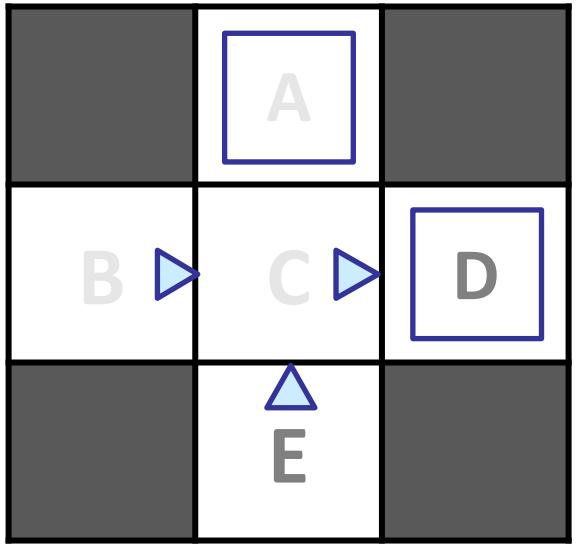
Value/utility of state c

$V^{\pi}(C) = ((9 + 9 + 9 + (-11))/4)$

$= 4$

Toy Example: Monte Carlo Method

Input Policy π



Assume: $\gamma = 1$

Observed Episodes (Training)

Episode 1

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 2

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 3

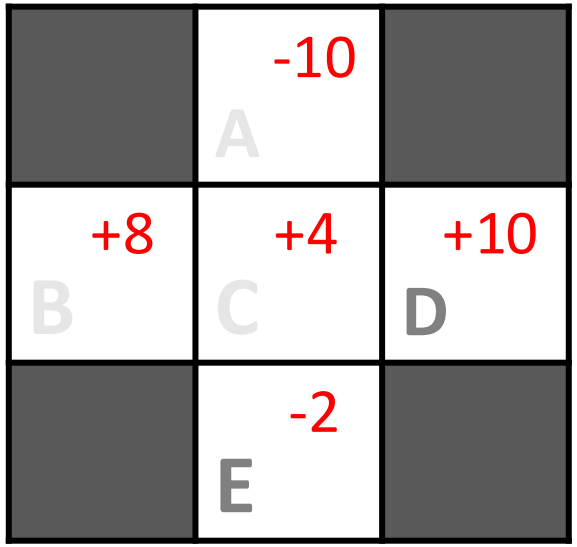
E, north, C, -1
C, east, D, -1
D, exit, x, +10

Episode 4

E, north, C, -1
C, east, A, -1
A, exit, x, -10

Value/utility of state c
$V^{\pi}(C) = ((9 + 9 + 9 + (-11))/4)$
$= 4$

Output Values



First-Visit Monte Carlo (FVMC) Policy Evaluation

Initialize the value function arbitrarily.

Loop over the episode from the end till the start.

Account for the discounting of the reward

Except if (unless) the state has been visited from time 0 till (t-1) append and average out the results.
I.e., only update the value estimate if this is the first visit to the state.

First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated
Initialize:
$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$
$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):
Generate an episode following π: $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$
$G \leftarrow 0$
Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
$G \leftarrow \gamma G + R_{t+1}$
Unless S_t appears in $S_0, S_1, \ldots, S_{t-1}$:
Append G to $Returns(S_t)$
$V(S_t) \leftarrow$ average$(Returns(S_t))$

Pseudo-code from Sutton and Barto (Reinforcement Learning) Ch 5 (Sec. 5.1)

Classical RL Techniques: TD-Learning

Recall the key relationships

Q value function: how can we express it? Take the action a on state s and then act optimally.

$$Q^*(s,a) = \sum_{s'} T(s,a,s')\left[\, R(s,a,s') + \gamma\, V^*(s') \,\right]$$

Optimal value function for a state s maximizes the Q-values of the state s and applicable actions a.

$$V^*(s) = \max_a Q^*(s,a)$$

Recursive way to write the value function (we have seen this definition before).

$$V^*(s) = \max_a \sum_{s'} T(s,a,s')\left[R(s,a,s') + \gamma V^*(s')\right]$$

- The utility of a state is the **immediate reward** for that state plus the **expected discounted utility of the next state** assuming that the agent is acting **optimally**.
- Note: The value function has some structure as they are modeling max. rewards that one can obtain from that state. The Bellman equations express that connection.

Why wait till the episode end?

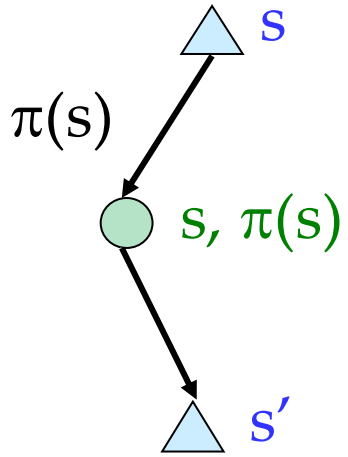
- Can we learn from each transition.
  - Update value function estimates of a state based on others
    - Adjust the value function estimate using the Bellman Equation relationship between the value function of successor states.
    - More data-efficient than a Monte Carlo method
- Setting
  - Can be used in an *episodic* or *infinite-horizon non-episodic* settings
    - Immediately updates the estimate of V(s) after each (s, a, s', r) tuple.

Temporal Difference Learning

"If one had to identify one idea that is central and novel to reinforcement learning, it would undoubtedly be temporal-difference (TD) learning", Sutton and Barto 2017

Temporal Difference (TD) Learning

- Temporal difference learning of values
  - Policy still fixed
  - Move values toward the value of the successor that is encountered
  - Keep a running average
  - Don't need to store all the experience to build models. It is a model-free approach.



Sample of V(s): $sample = R(s, \pi(s), s') + \gamma V^{\pi}(s')$ — What is observed

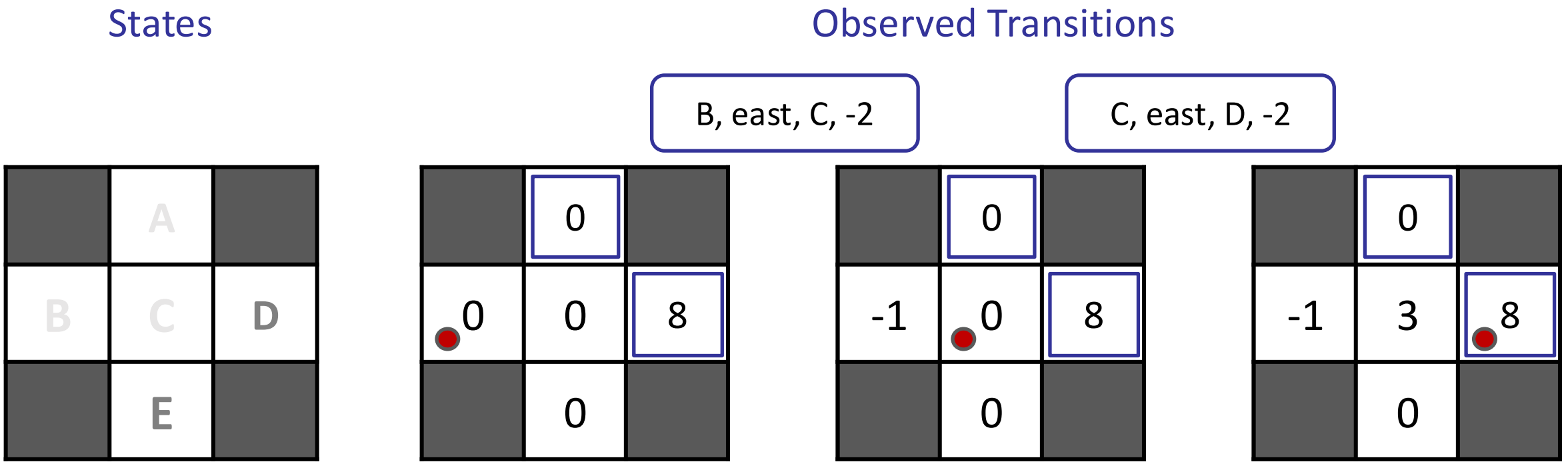
Update to V(s): $V^{\pi}(s) \leftarrow (1-\alpha)V^{\pi}(s) + (\alpha)sample$ — Modify the old value

Same update: $V^{\pi}(s) \leftarrow V^{\pi}(s) + \alpha(sample - V^{\pi}(s))$ — Sign and magnitude of the difference.

Temporal Difference Learning: Example

States

Observed Transitions



Assume: $\gamma = 1$, $\alpha = 1/2$

$$V^{\pi}(s) \leftarrow (1-\alpha)V^{\pi}(s) + \alpha\left[R(s,\pi(s),s') + \gamma V^{\pi}(s')\right]$$

Temporal Difference Learning

1. Initialize the value function, $V(s) = 0, \forall s$
2. Repeat as many times as wanted:
   (a) Pick a start state s for the current trial
   (b) Repeat for every time step t:
      i. Choose action a based on policy π and the current state s
      ii. Take action a, observed reward r and new state s'
      iii. Compute the TD error: $\delta \leftarrow r + \gamma V(s') - V(s)$
      iv. Update the value function:

$$V(s) \leftarrow V(s) + \alpha_s \delta$$

      v. $s \leftarrow s'$
      vi. If s' is not a terminal state, go to 2b

TD Learning (intuitively)

- Nudge our prior estimate of the value function for a state using the given experience.
- Shift the estimate based on the error in what we are experiencing and what estimate we had before
- Weighted by the learning rate.

TD Learning only gets us to the value function. Estimating the value function may be useful on its own.

Example 6.1: Driving Home Each day as you drive home from work, you try to predict how long it will take to get home. When you leave your office, you note the time, the day of week, the weather, and anything else that might be relevant. Say on this Friday you are leaving at exactly 6 o'clock, and you estimate that it will take 30 minutes to get home. As you reach your car it is 6:05, and you notice it is starting to rain. Traffic is often slower in the rain, so you reestimate that it will take 35 minutes from then, or a total of 40 minutes. Fifteen minutes later you have completed the highway portion of your journey in good time. As you exit onto a secondary road you cut your estimate of total travel time to 35 minutes. Unfortunately, at this point you get stuck behind a slow truck, and the road is too narrow to pass. You end up having to follow the truck until you turn onto the side street where you live at 6:40. Three minutes later you are home. The sequence of states, times, and predictions is thus as follows:

State	*Elapsed Time (minutes)*	*Predicted Time to Go*	*Predicted Total Time*
leaving office, friday at 6	0	30	30
reach car, raining	5	35	40
exiting highway	20	15	35
2ndary road, behind truck	30	10	40
entering home street	40	3	43
arrive home	43	0	43

The rewards in this example are the elapsed times on each leg of the journey.[1] We are not discounting ($\gamma = 1$), and thus the return for each state is the actual time to go from that state. The value of each state is the *expected* time to go. The second column of numbers gives the current estimated value for each state encountered.

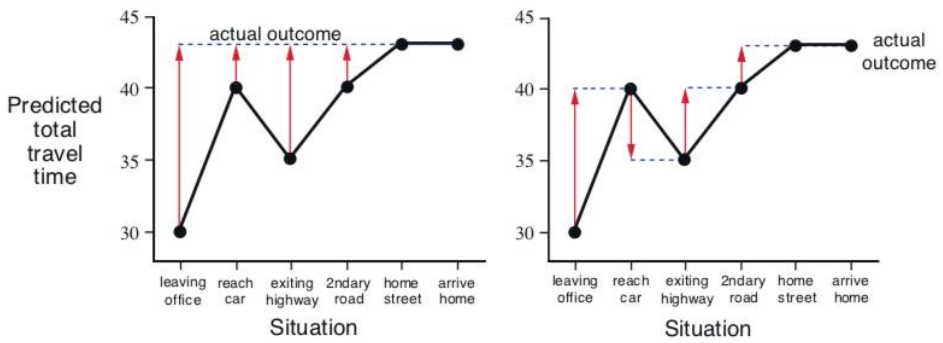


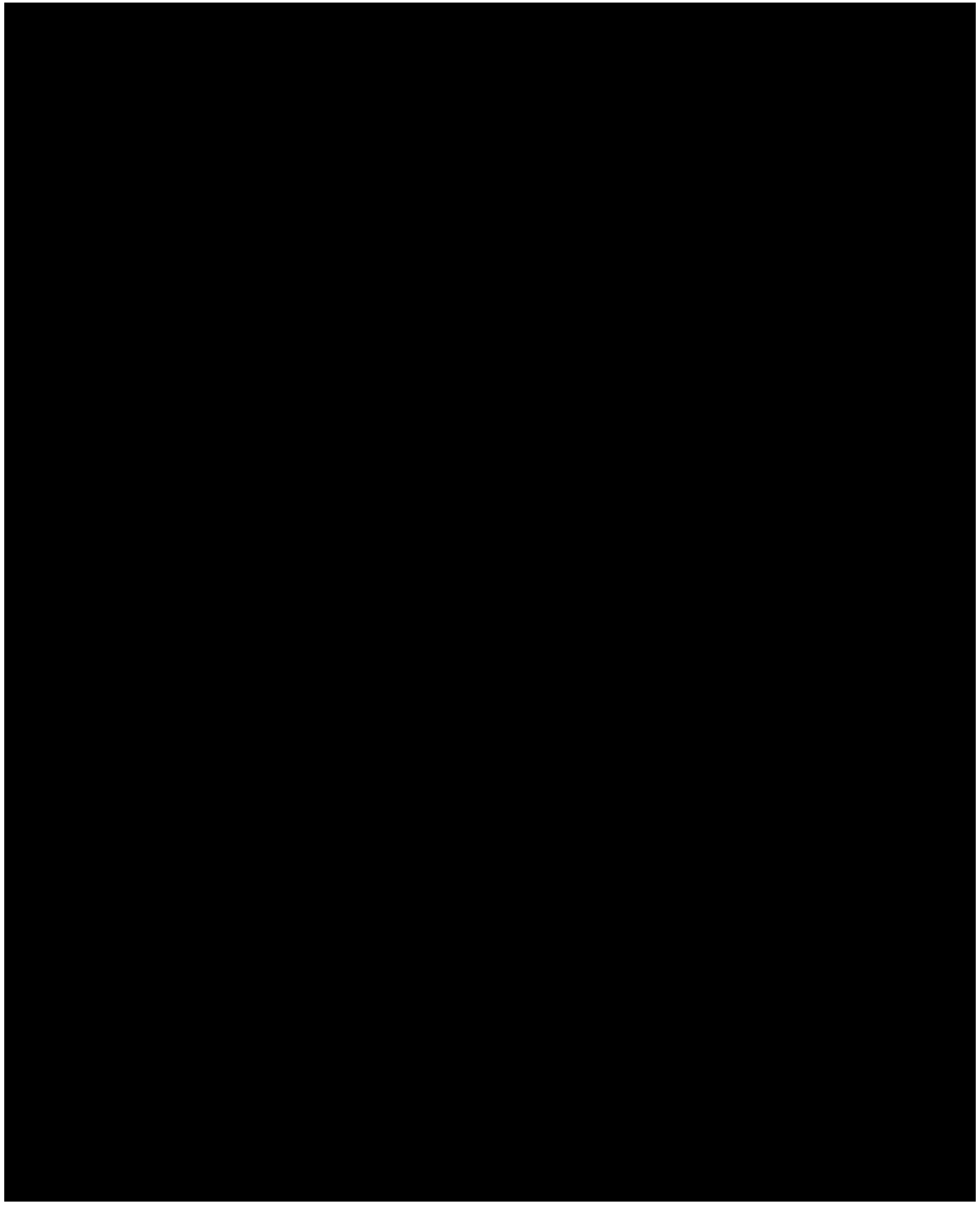
Figure 6.1: Changes recommended in the driving home example by Monte Carlo methods (left) and TD methods (right).

[1] If this were a control problem with the objective of minimizing travel time, then we would of course make the rewards the *negative* of the elapsed time. But because we are concerned here only with prediction (policy evaluation), we can keep things simple by using positive numbers.

Introduction to Q-Learning

Example: Learning to play a game

- The game of break out requires learning ways to hit the ball so as to break a large number of tiles.
- Actions – left, right.
- One needs to learn how to hit to maximize the breaking.
- We learn via experience, here we want to use RL for the same.



https://www.youtube.com/watch?v=V1eYniJ0Rnk

Recap: Q-Values

- Q*(s,a) is the expected utility starting in state s, taking action a and (thereafter) acting optimally.

Bellman Equation:

$$Q^*(s,a) = \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma \max_{a'} Q^*(s',a'))$$

Q-Value Iteration:

$$Q_{k+1}(s,a) \leftarrow \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma \max_{a'} Q_k(s',a'))$$

Learning the Q-function

- Core Idea: Learn Q(s, a) function from rollouts. Estimate the best that can be achieved by taking action a in state s.
- Derive the policy from the Q-function.

$$Q(s_t, a_t) = \mathbb{E}[R_t | s_t, a_t]$$

(state, action)

Ultimately, the agent needs a **policy $\pi(s)$,** to infer the **best action to take** at its state, s

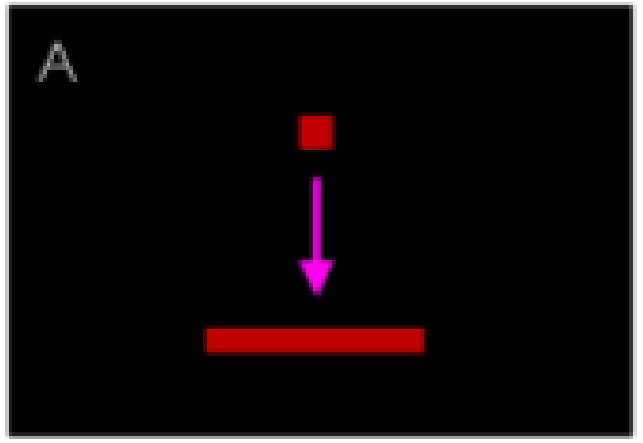
Strategy: the policy should choose an action that maximizes future reward

$$\pi^*(s) = \underset{a}{\operatorname{argmax}}\, Q(s, a)$$

Estimaation Q-functions is non-trivial

Example: Atari Breakout - Middle

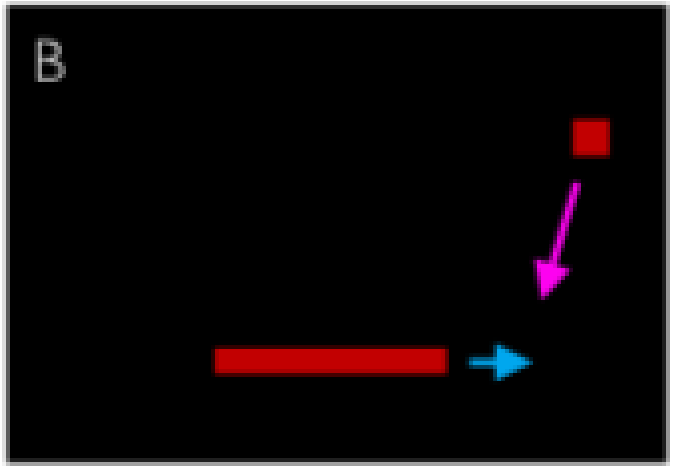
Can you guess the Q-value of this action?



Q-value for another action

Example: Atari Breakout - Side

Can you guess the Q-value of this action?



Q-learning idea: interact in the environment and build an estimate of the Q-function

From Value Iteration to Q-Value Iteration

- **Value Iteration**
  - Start with $V_0(s) = 0$
  - Given V_k, calculate the V_{k+1} for all states as:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V_k(s') \right]$$

- **Q-Value Iteration**
  - Start with $Q_0(s,a) = 0$
  - Given Q_k, calculate the depth Q_{k+1} q-values for all q-states:

$$Q_{k+1}(s,a) \leftarrow \sum_{s'} T(s,a,s') \underbrace{\left[R(s,a,s') + \gamma \max_{a'} Q_k(s',a') \right]}_{\text{Sample}}$$

Q-Learning

- Sample-based Q-value iteration

$$Q_{k+1}(s,a) \leftarrow \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma \max_{a'} Q_k(s',a') \right]$$

- Estimate Q(s,a) values as:
  - Receive a sample (s,a,s',r)
  - Consider your old estimate:
  - Consider your new sample estimate
  - Incorporate the new estimate into a running average:

$$Q(s,a)$$

$$sample = R(s,a,s') + \gamma \max_{a'} Q(s',a')$$

$$Q(s,a) \leftarrow (1-\alpha)Q(s,a) + (\alpha)\,[sample]$$

(Tabular) Q-Learning

Algorithm:

Start with $Q_0(s, a)$ for all s, a.

Get initial state s

For k = 1, 2, … till convergence

Sample action a, get next state s'

If s' is terminal:

$$\text{target} = R(s, a, s')$$

Sample new initial state s'

else:

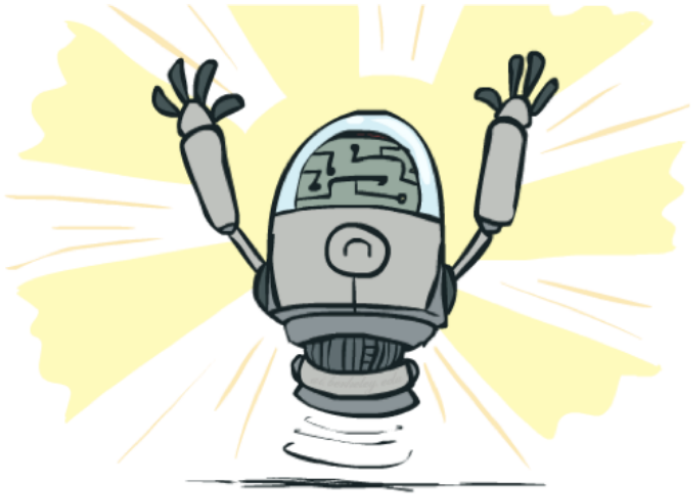
$$\text{target} = R(s, a, s') + \gamma \max_{a'} Q_k(s', a')$$

$$Q_{k+1}(s, a) \leftarrow (1 - \alpha) Q_k(s, a) + \alpha \, [\text{target}]$$

$$s \leftarrow s'$$

Q-learning properties

- Amazing result: Q-learning converges to optimal policy -- even if you're acting suboptimally!
- This is called off-policy learning
- Caveats:
  - You have to explore enough
  - You have to eventually make the learning rate small enough
  - … but not decrease it too quickly

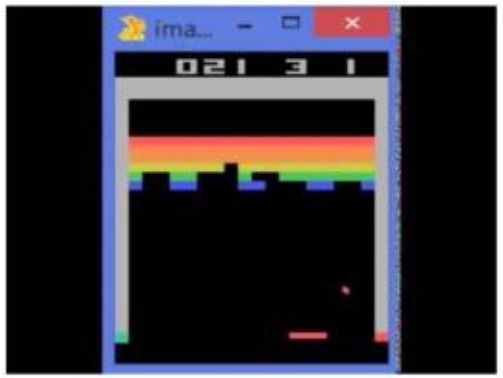


Problem of Generalization

- Problem when the state space is very large
  - Visiting all states is not possible at training time.
  - Memory issue: cannot fit the q-table in memory.
  - Need a succinct representation of the state.

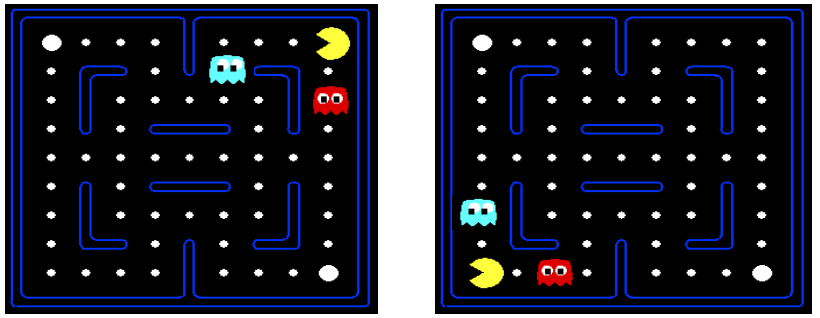
- Need for Generalization
  - Learn from *small number* of training states encountered in training
  - Generalize that experience to *new*, *similar situations* that not encountered before

- Feature-based Representations
  - Features or properties are functions from states to real numbers (often 0/1) that capture important properties of the state.
  - Example features that can be computed from the state
    - Distance to closest ghost
    - Distance to closest dot
    - Number of ghosts



State space is raw pixels, the state space size is large, cannot experience all states.

$$\left(256^{100\times200}\right)^{3}$$



The states are similar. But for Q-learning it will be a different entry. It does not know that in essence these states are the same.

Linear Value Functions

- Using a feature representation, a q function (or value function) can be expressed for any state using a set of weights:

$$V(s) = w_1 f_1(s) + w_2 f_2(s) + \ldots + w_n f_n(s)$$

$$Q(s,a) = w_1 f_1(s,a) + w_2 f_2(s,a) + \ldots + w_n f_n(s,a)$$

- Now the goal of Q-learning is to estimate these weights from experience. Once the weights are learned the resulting Q-values will hopefully be close to the true Q values.

- Main consequence:
  - A new state that shares features with a previously seen state will have the same Q-values.
- Disadvantage
  - If there are less features, actually different states (good and bad states) may start looking the same.

Approximate Q-learning

- Q-learning with linear Q-functions: $Q(s,a) = w_1 f_1(s,a) + w_2 f_2(s,a) + \ldots + w_n f_n(s,a)$

$$\text{transition} = (s, a, r, s')$$

Exact Q function updates

$$Q(s,a) \leftarrow Q(s,a) + \alpha \,[\text{difference}]$$

Approximate Q function updates

$$w_m \leftarrow w_m + \alpha \left[r + \gamma \max_a Q(s', a') - Q(s,a) \right] f_m(s,a)$$

"target" "prediction"

- Interpretation:
  - Adjust weights of active features.
  - If the difference is positive (what we get is higher than previous) and the feature value is 1 then the weight increases and the q value increases. No change if the feature value is 0.

The issue of exploration

Active Reinforcement Learning

- **Q-learning so far**
  - Q-learning allows action selection because we are learning the Q(s,a) function.
  - This means there is a policy that can be derived from the Q function learned.
  - Should the agent follow this policy exactly or should it explore at times?
- **Active RL**
  - The agent can select actions
  - Actions play two roles
    - A means to collect reward (exploitation)
    - **Help in acquiring a model of the environment (exploration)**
- **Exploration vs. Exploitation trade-off**
    - Act according to the current optimal (based on Q-Values)
    - Pick a different action to explore.
    - Example: a new tea stall opens in IIT (should you try the new one or stick to the old one? Goal is to maximize tea utility over time

Exploration Strategy

- **How to force exploration?**
- **An ε-greedy approach**
  - Every time step: either pick a random action or act on the current policy
  - With (small) probability **ε**, pick a random action
  - With (large) probability **1-ε**, act based on the current policy (based on the current Q –values in the table that the agent is updating)
- **What is the problem with ε-greedy?**
  - It takes a long time to explore.
  - Exploration is *not directed* towards states of which we have less information.

Exploration Functions

- How to **direct exploration** towards state-action pairs that are less explored?

- **Exploration function**
  - Trades off *exploitation* (preference for high values of u) vs. *exploration* (preference for actions that have not been tried out as yet).
  - f(u,n): Increasing in u and decreasing in n.

$$f(u, n) = u + k/n$$

- **What is the exploration function trying to achieve?**
  - The exploration function provides an optimistic estimate of the best possible value attainable in any state.
  - Makes the agent think that there is high reward propagated from states that are explored less.

Exploratory Q-Learning

- **In Q-learning**
  - Explicitly encode the value of exploration in the Q-function
- **Exploratory Q-Learning**

$$Q(s,a) \leftarrow_\alpha R(s,a,s') + \gamma \max_{a'} f(Q(s',a'), N(s',a'))$$

Modified update

- **Effects**
  - The lower the N(s', a') is the higher is the exploration bonus.
  - The exploration bonus makes those states favorable which lead to unknown states (propagation).
  - Will have a cascading effect when an exploration action is there.

Takeaways

- Learning from reinforcement applies to problems where there is no knowledge of which states have good rewards and how the actions lead to new states,

- Various approaches for performing RL: estimating a model, estimating the value function or directly optimizing the policy (not covered yet). Variation with whether we learn from the whole episode or from each transition.

- Data is essentially experience of the agent. There is evaluative feedback not prescriptive feedback. RL is different from supervised learning.

- Fundamental tradeoff between exploration and exploitation. Generalization is a challenge in large state spaces.